

Feature Pipeline Report

NBA Player Stats Prediction System Generated: 2026-03-27

This report describes the preprocessing feature pipeline used by the project. It covers raw data ingestion, feature engineering, feature selection, and the handoff into the active CatBoost + Transformer training stack.

Executive Summary

- Raw player and team CSVs are merged into a single player-game table.
- FeatureEngineer builds a leakage-safe feature set using 15 phases.
- The active path is chronological, shifted, and training-only where needed.
- GPU feature engineering exists, but the default constructors use CPU mode.

One-Line Flow

```
data/nba_players.csv  
data/nba_games.csv  
  |  
  v  
DataLoader.merge_datasets()  
  |  
  v  
FeatureEngineer.create_features()  
  |  
  v  
feature selection and cleaning  
  |  
  v  
TrainingPipeline.prepare_data()  
  |  
  v  
CatBoost + Transformer models
```

Source Map

- src/preprocessing/data_loader.py: CSV loading, minutes parsing, merge logic, and opponent defense derivation.
- src/preprocessing/feature_engineer.py: main 15-phase feature pipeline and the optional GPU rolling helper.
- src/pipeline/data_pipeline.py: validation wrapper, target capping, sample weights, and leakage-safe feature selection.
- src/training/pipeline.py: active training orchestration used by train.py.
- src/preprocessing/features/base.py and src/preprocessing/features/rolling.py: modular feature-group scaffold.

Data Ingestion and Normalization

DataLoader does the following:

- loads data/nba_players.csv and data/nba_games.csv
- coerces numeric stat columns such as PTS, REB, AST, FGA, FTA, and TOV
- parses MIN values from either decimal or MM:SS formats
- converts GAME_DATE to datetime
- drops rows missing PTS, REB, AST, or MIN
- sorts player rows by PLAYER_ID and GAME_DATE

It then self-joins the team game table on GAME_ID to infer opponent stats and to build rolling team defense fields such as TEAM_DEF_OPP_PTS_ALLOWED_ROLL_10. Those team-level fields are merged back into each player row together with an OPPONENT_ID mapping.

FeatureEngineer Control Flow

The active feature builder is a single class rather than the modular FeatureGroup API. The group classes in `src/preprocessing/features/` are present, but they are not wired into the main pipeline.

The control flow is:

- validate `PLAYER_ID` and `GAME_DATE`
- sort by `PLAYER_ID` and `GAME_DATE`
- convert `GAME_DATE` to datetime and drop invalid dates
- run the 15 feature phases
- drop duplicate columns, temporary `_tmp_` columns, inf values, and NaNs
- drop rows that still lack `ROLL_PTS_AVG_5`
- execute league-ranking and target-encoding phases only when `is_training=True`

The pipeline has two rolling implementations:

- GPU rolling via PyTorch when `use_gpu=True` and the frame is large enough
- vectorized CPU rolling with joblib threads in the normal path

`create_features_chunked` keeps an entire player's history in one chunk so rolling statistics remain chronological inside each player group.

Phase map:

- 1 Rolling features
- 2 Efficiency features
- 3 Momentum features
- 4 Contextual features
- 5 Matchup history
- 6 Advanced scoring
- 7 Pace adjusted stats
- 8 Teammate features
- 9 Opponent strength
- 9b Defensive matchup features
- 10 Interaction features
- 11 Bayesian estimates
- 12 Seasonality
- 13 Advanced custom
- 14 League ranking
- 15 Target encoding

Phases 1-5

1. Rolling Features

Rolling stats are built on shifted history so the current game is never part of the input. Windows: 3, 5, 10, 20, 50. Stats: mean, std, min, max. Range features are added for 10- and 20-game windows.

2. Efficiency Features

- TS%, eFG%, 3PT%, AST/TOV
- per-minute outputs for PTS, REB, AST

These are computed from shifted rolling sums over the player's prior games.

3. Momentum Features

- EWMA spans: 3, 5, 10, 20
- season-to-date expanding mean
- short-minus-long trends
- hot/cold streak flags

The code uses the same shifted history, so momentum is always backward-looking.

4. Contextual Features

- IS_HOME from MATCHUP
- DAYS_SINCE_LAST, REST_DAYS, IS_B2B
- MINS_LAST_3 and MINS_LAST_7
- FATIGUE_SCORE from rest and recent minutes

The fatigue score is a simple weighted blend of recent load, back-to-backs, and short rest.

FATIGUE_SCORE:

$$0.4 * \text{MINS_LAST_3} / 100 + 0.3 * \text{IS_B2B} + 0.3 * (4 - \text{REST_DAYS})$$

5. Matchup History

For each PLAYER_ID and OPPONENT_ID pair the code adds:

- VS_OPP_{stat}_AVG
- VS_OPP_{stat}_RECENT

This creates career and recent performance context against the current opponent.

Phases 6-10

6. Advanced Scoring Features

This phase pushes the feature set beyond generic rolling stats:

- ROLL_USG_PCT_10 from shifted possession share
- ROLL_REB_OPPORTUNITY_* from opponent misses
- ROLL_3PT_FREQ_* and ROLL_FT_RATE_10 from shot profile
- ROLL_PTS_SHARE_10 from player versus team scoring share
- ROLL_TS_PCT_MOMENTUM_* from recent true shooting

7. Pace Adjusted Stats

- EST_POSS estimates team possessions from team box-score components
- TEAM_PACE_10 is a 10-game shifted team pace average
- PACE_FACTOR scales the context to a league-average 100 pace baseline

8. Teammate Features

- STAR_TEAMMATE_OUT is a default availability flag
- TEAM_{stat}_AVG_10 captures shifted team context
- PLAYER_{stat}_SHARE tracks share of team production

These features let the model distinguish a player's role from the team total.

9. Opponent Strength

RELATIVE_OPP_DEF_{stat} normalizes opponent defense against league averages. If the defensive column is missing, the code falls back to 1.0 so the feature pipeline keeps moving.

9b. Defensive Matchup Features

The defensive matchup block is one of the richest parts of the pipeline. It adds:

- DEF_MATCHUP_{stat}_IMPACT
- OPP_DEF_RATING and DEF_DIFFICULTY
- DEF_OVERAGE_{stat}
- DEF_MATCHUP_TREND_{stat}
- DEF_MATCHUP_HOME_ADJ and DEF_MATCHUP_AWAY_ADJ
- OPP_DEF_RANK and QUALITY_DEF_AVOIDANCE

This block turns opponent defense into a direct, player-specific signal.

10. Interaction Features

- stat potential = rolling production times opponent defense
- B2B_IMPACT_FACTOR blends rest and recent scoring
- B2B_MIN_IMPACT uses recent minutes on back-to-backs

These are lightweight interaction terms built from already safe inputs.

Phases 11-15

11. Bayesian Estimates

Each target stat gets a shrinkage estimate:

- player expanding mean
- global mean
- weight based on games played

The result is a stabilized prior-like estimate that is less noisy early in the season.

12. Seasonality

Fourier features encode day-of-year seasonality:

- SEASON_SIN_1, SEASON_COS_1
- SEASON_SIN_2, SEASON_COS_2

These let the model learn year-round trends without a hard calendar bucket.

13. Advanced Custom Features

This block adds several higher-signal features:

- ROLL_USG_PCT_10 and PACE_ADJ_USAGE
- SOS_OPP_DEF_5 for strength of schedule
- EFF_Z_SCORE for hot-hand or slump detection
- ROLL_FANTASY_PTS_10 when fantasy points are available

14. League Ranking

League percentile features are only added in training mode. The code uses a trailing global window of 2000 rows with 500 minimum periods and computes LEAGUE_PCT_{stat} as the current stat's percentile in that window.

15. Target Encoding

Target encodings are also training-only. The code builds player and team encodings with smoothing and expanding history, producing features such as PTS_PLAYER_TE and PTS_TEAM_TE. This is one of the main leakage-sensitive stages, so it is never run for inference-only feature generation.

Feature Selection, Cleaning, and Handoff

DataPipeline and TrainingPipeline both protect against leakage, but they do it with slightly different selectors.

DataPipeline.select_features():

- excludes IDs, dates, targets, and obvious leak columns
- keeps only numeric columns
- then whitelists safe prefixes, suffixes, and explicit contextual features

TrainingPipeline._select_features():

- uses the same exclusions
- accepts the same rolling, EWMA, matchup, pace, Bayesian, and defense families
- also accepts generic numeric columns that survive the exclude list

This is the selector used by train.py.

Target preprocessing:

- preprocess_targets caps each target at the configured percentile per player
- missing player caps fall back to the global percentile
- cleaned targets are written to *_CLEAN columns

Temporal weighting:

- sample weights are computed as $\exp(-\lambda \cdot \text{days_ago})$
- values are clipped to the [0.1, 1.0] range

Training handoff:

- train.py loads and merges raw data
- FeatureEngineer.create_features() builds the full feature table
- TrainingPipeline.prepare_data() sorts by GAME_DATE and splits fit/validation/test
- TrainingPipeline.train() saves feature_cols.pkl and blend_weights.pkl
- ModelManager and the simulator reload feature_cols.pkl for inference

Config values that shape the pipeline:

- training.targets = PTS, REB, AST, STL, BLK, TOV
- training.test_split_date = 2025-01-01
- training.temporal_decay_lambda = 0.023
- training.outlier_percentile = 0.99
- features.rolling_windows = 3, 5, 10, 20, 50
- features_ext.target_encoding_smoothing = 20

Validation and Observations

Tests currently cover:

- FeatureEngineer import, initialization, custom windows, empty input, missing columns, and a leakage smoke check
- DataPipeline initialization, temporal weights, target preprocessing, and feature selection
- configuration loading and the training pipeline path

Operational observations:

- The FeatureGroup layer is a useful scaffold, but it is not part of the active pipeline yet
- GPU feature engineering is optional and off by default in the main constructors
- Some phases fall back to safe defaults when a supporting column is missing
- The pipeline is strong on leakage control, but the implementation is monolithic and could later be decomposed through the FeatureGroup abstraction

Relevant files:

- src/preprocessing/data_loader.py
- src/preprocessing/feature_engineer.py
- src/pipeline/data_pipeline.py
- src/training/pipeline.py
- src/preprocessing/features/base.py
- src/preprocessing/features/rolling.py
- tests/test_preprocessing/test_feature_engineer.py
- tests/test_pipeline/test_data_pipeline.py

Conclusion: The feature pipeline is a carefully layered, leakage-aware preprocessing stack that turns raw NBA player and team logs into a rich tabular feature space for the active CatBoost + Transformer models.